

JEGYZŐKÖNYV

Modern adatbázis rendszerek MSc

LDAP

Készítette: Szalóczy Krisztián

Neptunkód: Y4O4X0

Dátum: 2026.05.13

Tartalom jegyzék

1.	A feladat leírása	4
1.1	Feladatok tervezésének lépései.....	5
1.2	Futtatás eredménye	6
2.	A feladat leírása	11
2.1	A feladat elmélete.....	11
2.2	Gyakorlati feladatok	12

Témakör

Ez a dokumentum betekintést nyújt az LDAP lehtőségeibe, és egy szerver implementáció, az OpenLDAP projekthez tartozó *slapd* használatába a teljesség igénye nélkül. Nem vállalkoztam arra, hogy minden angol terminológiából ismert szakkifejezést magyarítok.

További dokumentációk elérhetőek itt: <http://www.openldap.org>. Ez a dokumentum tárgyal olyan témaköröket, melyeket itt nem tárgyalunk (pl. adatbázis replikáció, elosztott adatbázis üzemeltetése, TLS konfigurálása).

LDAP alapok

Mi az az LDAP?

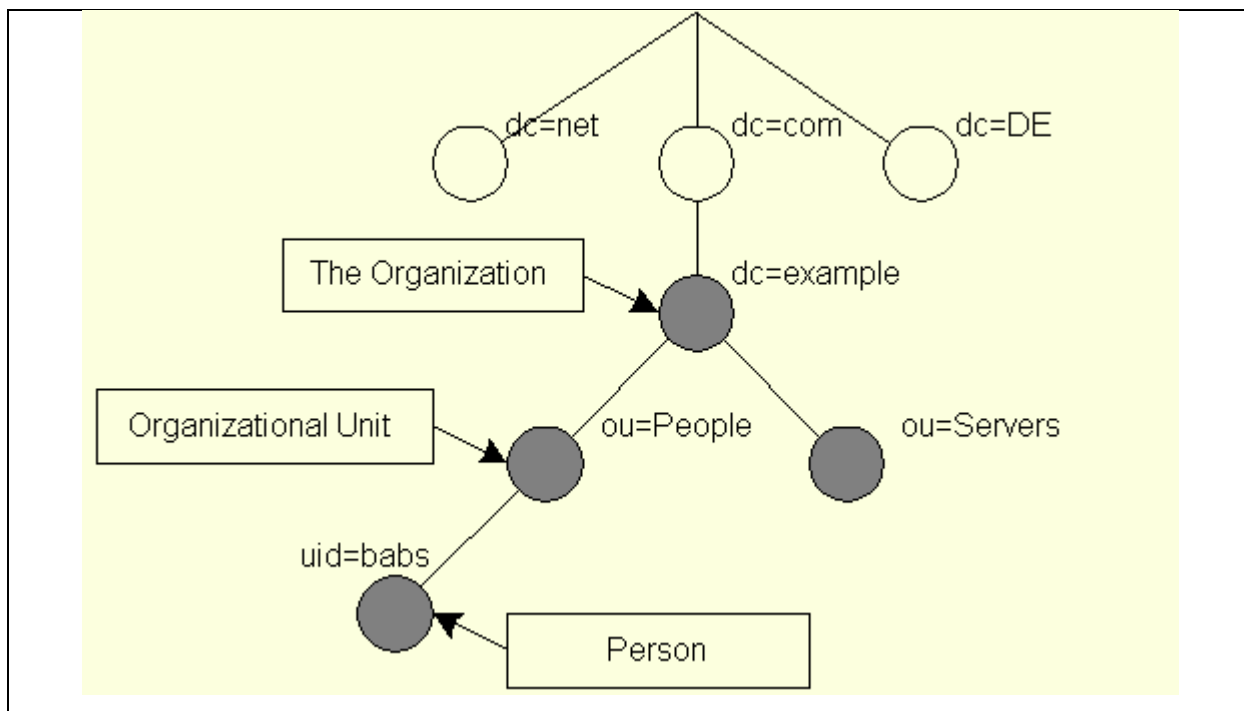
Az LDAP a Lightweight Directory Access Protocol rövidítése. Mint a neve mutatja, az LDAP nem több egy protokollnál, nem egy konkrét szoftverről van tehát szó. Ez a protokoll directory szolgáltatások elérését szabályozza.

Mi az a directory szolgáltatás?

Az angol *directory service* egy olyan speciális adatbázist takar, mely keresésre van optimalizálva, ennek megfelelően olyan esetekben célszerű ilyet használni, ahol kevés a módosítás, és nagy számú, gyors lekérdezésekre van szükség.

Az információ egy faszerű szerkezetben tárolódik, és minden csúcsában bejegyzések (entry) szerepelnek. Egy bejegyzésnek van típusa, amely meghatározza, hogy milyen attribútumai lehetnek. Minden egyes ilyen bejegyzésre egyértelműen hivatkozhatunk a bejegyzés DN-jével (Distinguished Name), mely lényegében a fában a csúcsához vezető utat írja le.

Illusztrációként hadd vegyem kölcsön az OpenLDAP dokumentációból ezt a képet:



Ebben az esetben a dc (domain component) bejegyzések az Internet domain nevek elrendeződését veszik alapul. A példában az example.com site hierarchiájának egy részlete látható. A legmélyebben fekvő csúcs DN-je: *uid=babs,ou=People,dc=example,dc=com*

1. A feladat leírása

A feladat egy LDAP API létrehozása java programozási nyelven ami egy docker konténerben futó LDAP szervezethez csatlakozik. A kapcsolat kialakítása után a főbb műveletek végrehajtása mint az ldapadd, ldapmodify, ldapsearch, ldapdelete lesz, hogy megértsük hogyan is működnek ezek az operációk. Ehhez szükségünk lesz Maven-re ami a java egyik csomag kezelője (package manager). Továbbá a novell.ldap könyvtárat fogjuk használni ami biztosítja a főbb osztályokat és interfészeket a kliens és ldap server összekapcsolásához. Ami az LDAP server felállítását illeti szüksége lesz Docker-re. Windows esetén Docker Desktop remek

segítséget nyújt a konténerekkel való munkához. Egy online elérhető docker image-t fogunk használni ami egy pár soros paranccsal elindítható.

1.1 Feladatok tervezésének lépései

A feladatokat gyakorlat orientáltak – tehát maga a környezet kialakítás, Maven megismerése és a hozzá tartozó pom.xml illetve a konténerekkel való kommunikáció a főbb docker parancsok használata.

Az LDAP szerver feladat célja, hogy egy központi címtárszolgáltatás működését megértsük és gyakorlatban kipróbáljuk. A megvalósítás során egy Docker konténerben futó LDAP szervert használok, amely lehetővé teszi, hogy a felhasználók és egyéb címtárbejegyzések strukturált formában legyenek tárolva. Ezzel a feladattal megérthető, hogyan működik egy hierarchikus adatbázis-szerű rendszer, amelyet leggyakrabban hitelesítésre és szervezeti adatok kezelésére használnak.

Az LDAP add (ldapadd) feladat célja új bejegyzések létrehozásának megértése a címtárban. Ezzel a művelettel felhasználókat, csoportokat vagy egyéb objektumokat tudunk hozzáadni LDIF fájlok segítségével. Ebből megérthető, hogyan kerülnek strukturált adatok az LDAP szerverbe, és hogyan épül fel a címtár hierarchiája.

Az LDAP modify (ldapmodify) feladat célja a meglévő bejegyzések módosításának elsajátítása. Ennek segítségével például felhasználói adatok, jelszavak vagy attribútumok frissíthetők anélkül, hogy az adott bejegyzést újra kellene létrehozni. Ezzel a feladattal megérthető, hogyan kezelhetőek dinamikusan a címtárban tárolt adatok és hogyan történik azok karbantartása.

Az LDAP search (ldapsearch) feladat célja a címtárban történő lekérdezések megértése. Ezzel a művelettel különböző szűrők alapján lehet adatokat keresni, például felhasználónevek vagy csoportok szerint. Ez segít megérteni, hogyan

történik az adatok gyors lekérdezése egy hierarchikus adatbázisban, és hogyan használható az LDAP autentikációs folyamatokban.

A Docker feladat célja a rendszer futtatási környezetének megértése és reprodukálhatóságának biztosítása. Az LDAP szerver egy előre elkészített Docker image-ből kerül indításra, amely biztosítja, hogy a rendszer minden környezetben azonos módon működjön. A Docker parancsok segítségével megérthető a konténerek indítása, kezelése és hálózati elérése, például a portok megnyitása és a futó konténerek ellenőrzése.

A Maven feladat célja a Java alkalmazás buildelési és függőségkezelési folyamatának megértése. A Maven segítségével automatizáltan kezelhetők a projekt függőségei, például az LDAP kliens könyvtár, valamint egyszerűen elvégezhető a projekt fordítása és csomagolása. Ezzel megérthető, hogyan lehet egy Java projektet strukturáltan felépíteni és egységesen kezelni.

1.2 Futtatás eredménye

Java kliens létrehozása és novell.ldap könyvtár használata:

```
package org.example;
import com.novell.ldap.*;
import java.util.Arrays;
public class Main {
    static void main(String[] args) {
        LDAPConnection conn = new LDAPConnection();
        try {
            conn.connect("172.20.112.1", 389);

            conn.bind(
                LDAPConnection.LDAP_V3,
```

```
        "cn=admin,dc=demo,dc=local",  
        "admin"  
    );  
    System.out.println("Sikeres bind!");  
    conn.disconnect();  
    } catch (LDAPException e) {  
        e.printStackTrace();  
    }  
}  
}
```

1.ábra: java kliens

Függőségek: novell.ldap - pom.xml:

```
<dependencies>  
    <dependency>  
        <groupId>com.novell.ldap</groupId>  
        <artifactId>jldap</artifactId>  
        <version>4.3</version>  
        <scope>compile</scope>  
    </dependency>  
</dependencies>
```

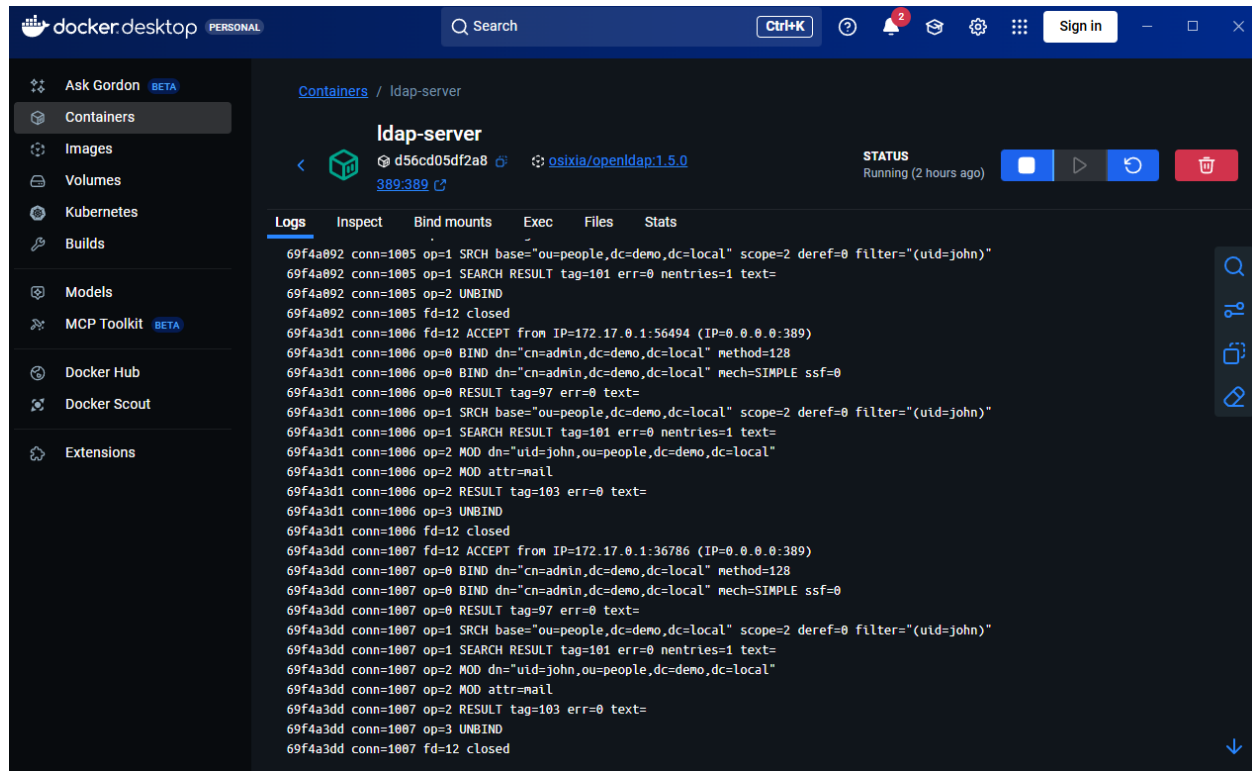
2.ábra: pom.xml

LDAP szerver elindítása parancssorból:

```
Docker run -d --name ldap-server -p 389:389  
-e LDAP_ORGANISATION="Demo"  
-e LDAP_DOMAIN="demo.local"  
-e LDAP_ADMIN_PASSWORD="admin"  
osixia/openldap:1.5.0
```

3.ábra: docker parancs

Futó konténer – Docker desktopban:



4.ábra: Futó ldap konténer

Hasznos docker parancsok:

- docker ps – futó konténerek listázása
- docker images – image-ek listázása
- docker run <image> - konténer indítása image-ből
- docker run -d <image> - háttérben futtatás
- docker stop <container> - konténer leállítása
- docker rm <container> - konténer eltávolítása
- docker logs <container> - logok megtekintése

Java API címtárstruktúra kialakítása:

```
LDAPAttributeSet attrs = new LDAPAttributeSet();
    attrs.add(new LDAPAttribute("objectClass", "organizationalUnit"));
    attrs.add(new LDAPAttribute("ou", "people"));

    String dn = "ou=people,dc=demo,dc=local";
    conn.add(new LDAPEntry(dn, attrs));
    String dn = "uid=john,ou=people,dc=demo,dc=local";

    LDAPAttributeSet attrs = new LDAPAttributeSet();
    attrs.add(new LDAPAttribute("objectClass", "inetOrgPerson"));
    attrs.add(new LDAPAttribute("cn", "John Doe"));
    attrs.add(new LDAPAttribute("sn", "Doe"));
    attrs.add(new LDAPAttribute("uid", "john"));
    attrs.add(new LDAPAttribute("userPassword", "1234"));

    conn.add(new LDAPEntry(dn, attrs));

    String base = "ou=people,dc=demo,dc=local";
    String filter = "(uid=john)";
    int scope = LDAPConnection.SCOPE_SUB; // teljes részfa

    LDAPSearchResults results = conn.search(base, scope, filter, null, false);

    while(results.hasMore()) {
        LDAPEntry entry = results.next();
        System.out.println("DN: " + entry.getDN());

        LDAPAttributeSet attrs = entry.getAttributeSet();
        for (Object o : attrs) {
            LDAPAttribute attr = (LDAPAttribute) o;
            System.out.println(attr.getName() + " = " +
                Arrays.toString(attr.getStringValueArray()));
        }
    }
```

```
}  
}
```

5.ábra: Struktúra módosítása

Lekérdezés végrehajtása:

```
String base = "ou=people,dc=demo,dc=local";  
String filter = "(uid=john)";  
int scope = LDAPConnection.SCOPE_SUB; // teljes részfa  
LDAPSearchResults results = conn.search(base, scope, filter, null, false);  
  
while(results.hasMore()) {  
    LDAPEntry entry = results.next();  
    System.out.println("DN: " + entry.getDN());  
  
    LDAPAttributeSet attrs = entry.getAttributeSet();  
    for (Object o : attrs) {  
        LDAPAttribute attr = (LDAPAttribute) o;  
        System.out.println(attr.getName() + " = " +  
            Arrays.toString(attr.getStringValueArray()));  
    }  
}
```

6.ábra: Lekérdezés

Lekérdezés eredménye:

```
DN: uid=john,ou=people,dc=demo,dc=local  
uid = [john]  
cn = [John Doe]  
sn = [Doe]  
objectClass = [inetOrgPerson]  
userPassword = [1234]
```

7.ábra: Search eredménye

Módosítás végrehajtása:

```
LDAPModification mod = new LDAPModification(  
    LDAPModification.REPLACE,  
    new LDAPAttribute("mail", "newmail@demo.local")  
);  
  
conn.modify(dn, mod);  
System.out.println("Sikeres módosítás!");
```

8.ábra: Módosítás

2. A feladat leírása

2.1 A feladat elmélete

A feladat célja egy LDAP alapú címtárszolgáltatás gyakorlati megvalósítása és az alapvető címtárműveletek (létrehozás, keresés, módosítás, törlés) működésének megértése. A rendszer egy Docker konténerben futó LDAP szerverre épül, amelyhez egy Java alkalmazás LDAP kliensként kapcsolódik.

Az első lépés a címtárstruktúra kialakítása volt, amely során létrejött az ou=people,dc=demo,dc=local szervezeti egység. Ez biztosítja a felhasználók logikai csoportosítását és a hierarchikus adatkezelés alapját. Ezt követően került sor egy felhasználói bejegyzés (uid=john) létrehozására, amely a címtárban tárolt objektumok alapját képezi.

A keresési művelet célja az LDAP címtárban tárolt adatok lekérdezésének megértése. Ennek során különböző szűrők segítségével lehet meghatározott bejegyzéseket keresni, például felhasználónév alapján. Ez a művelet alapvető szerepet játszik az adatok elérésében és a későbbi autentikációs folyamatok előkészítésében.

A módosítási művelet (modify) lehetővé teszi a már meglévő LDAP bejegyzések frissítését, például attribútumok (e-mail cím, jelszó vagy egyéb adatok) megváltoztatását. Ez biztosítja a címtárban tárolt adatok karbantarthatóságát és dinamikus kezelését.

A törlési művelet (delete) az LDAP címtárban lévő bejegyzések eltávolítását valósítja meg. Ennek segítségével teljes objektumok törölhetők a hierarchiából, így a címtár tartalma tisztítható és karbantartható marad. A négy alpművelet együtt biztosítja az LDAP rendszerekben használt teljes CRUD (Create, Read, Update, Delete) funkcionalitást.

A rendszer futtatása Docker alapú környezetben történik, ahol az LDAP szerver egy előre elkészített image-ből kerül indításra. A Docker biztosítja a reprodukálható környezetet, a könnyű indítást és a portokon keresztüli elérhetőséget. A Java alkalmazás Maven segítségével kerül felépítésre, amely kezeli a függőségeket és a projekt fordítását, így biztosítva a strukturált fejlesztési környezetet.

2.2 Gyakorlati feladatok

Készítsem egy Java API ami egy konténerben futó LDAP szerverhez kapcsolódik. Használja a novell.ldap könyvtárat amit maven segítségével huzzón be a java projektjébe. Az ldap szerver elindításához az osixia/openldap image-et futtasa egy konténerben.

a) Valósítsa meg a következő feladatokat (java API-ból):

1. LDAP címtár alapstruktúrájának kialakítása pl.:

ou=people,dc=demo,dc=local - ami felhasználók logikai tárolására szolgál

2. Vigyen fel egy felhasználói bejegyzést
(`uid=john,ou=people,dc=demo,dc=local`), amely alap attribútumokat (pl. név, jelszó, e-mail) tartalmaz.
3. Végezzen LDAP keresést kell végrehajtani a `ou=people,dc=demo,dc=local` alatti részében, a (`uid=john`) szűrőfeltétellel.
4. Módosítsa az `uid=john,ou=people,dc=demo,dc=local` felhasználó adatait, például az e-mail cím vagy más attribútum cseréjével.
5. Törölje a `uid=john,ou=people,dc=demo,dc=local` felhasználót